CS & IT ENGINEERING

Operators

C Programming

Lecture No. 04

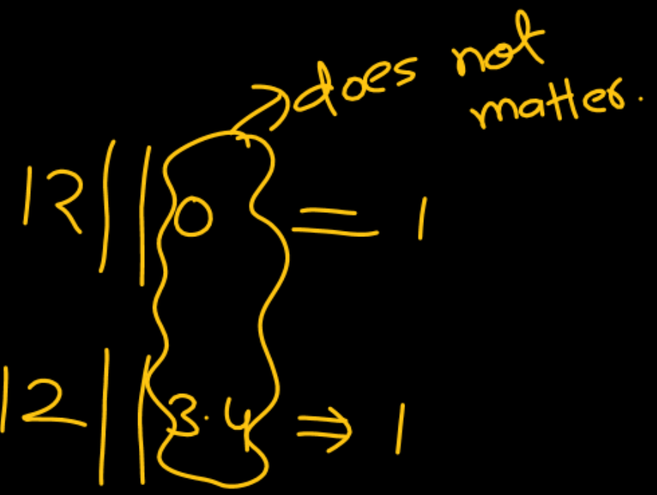
Pankaj Sir

'Operators

result of 12 || ▭

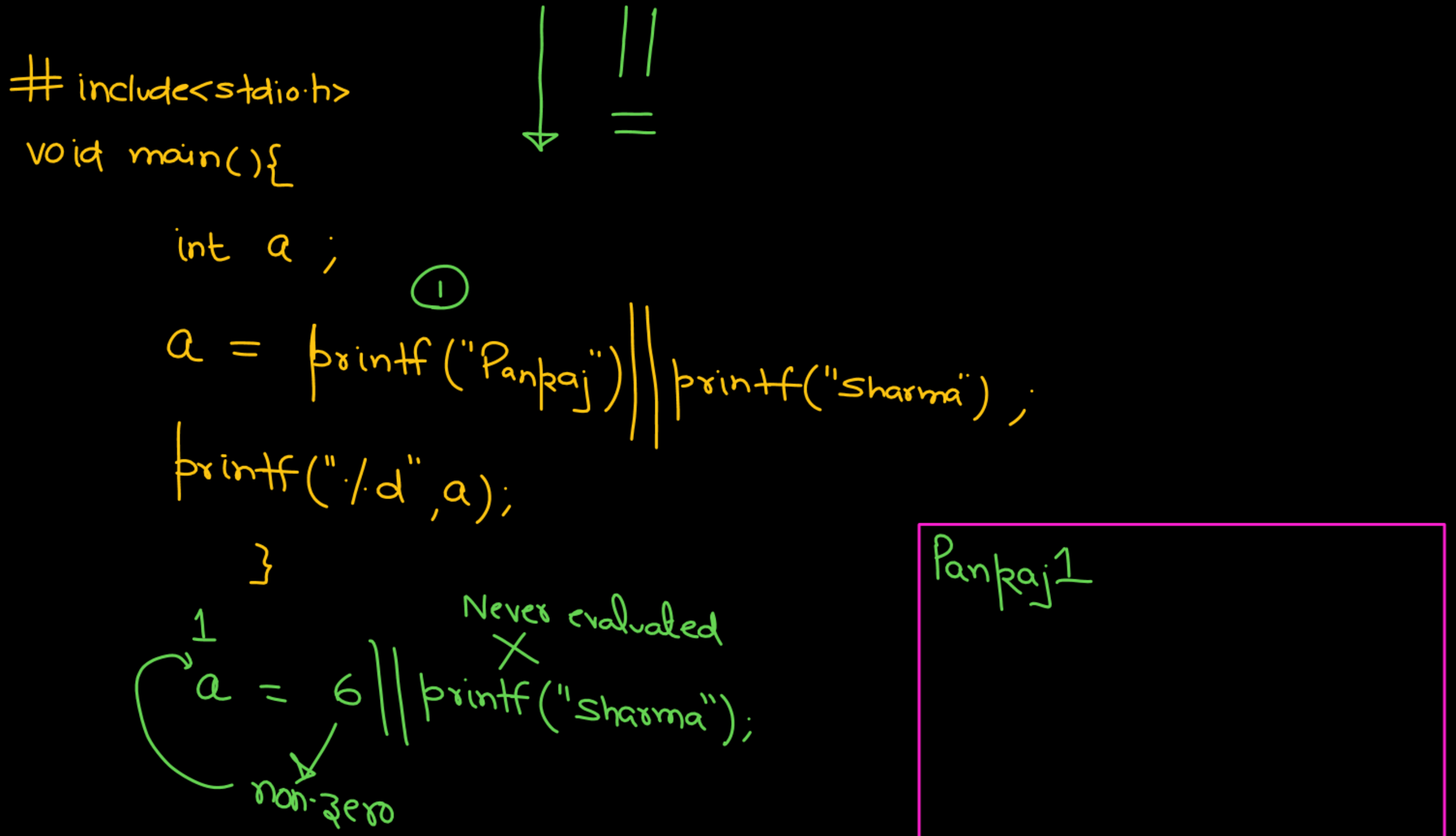
do we need it?

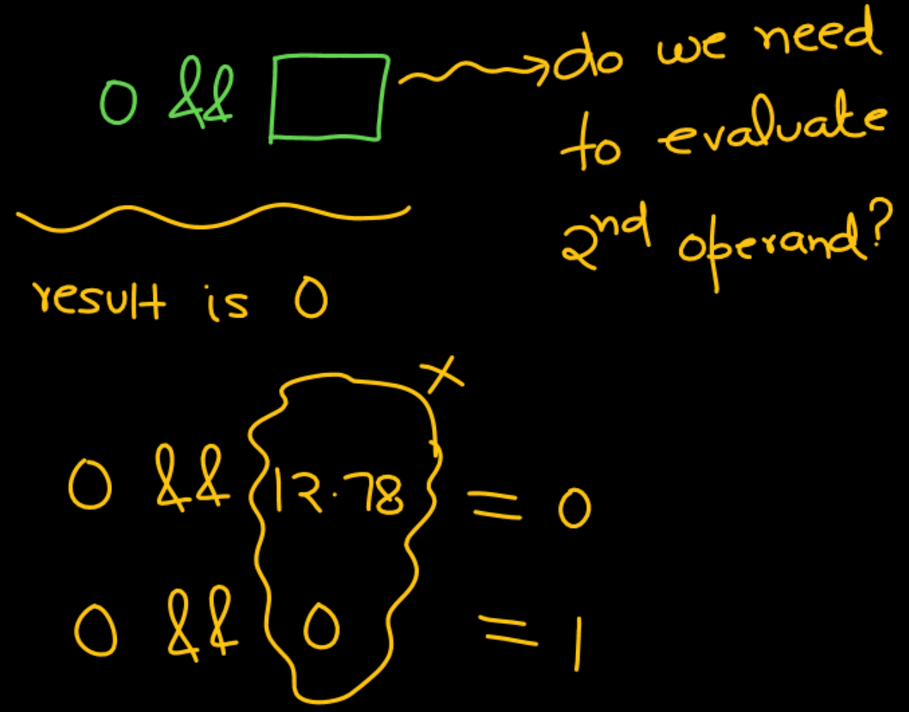
First operand non-zero then ⇒ result is 1 and we do not need 2nd operand to get the result.



Short-circuit Evaluation:

If the first operand for logical operator is non-zero (True), second operand will not evaluated.


#include<stdio.h>
void main(){
int a ;
a = printf("Pankaj") || printf("sharma") ;
printf("%d",a);
}
1
Never evaluated
a = 6 || printf("sharma");
non-zero
Pankaj1



If the first operand is zero, then second operand will not evaluated.

1.

```
int a;
a = printf("") && printf("Pankaj");
printf("%d", a);
```

a = 0 (0)

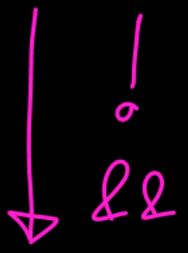
a = 0 && printf("Pankaj");

Not evaluated

```
0
```

3.

```
int a;
a = !printf("Pankaj") && printf("sharma");
printf("%d",a);
```



!6 && printf("sharma");

a = 0 && printf("Sharma");

0

Never evaluated

Pankaj0

```
a = printf("!Pankaj") && printf("Sharma");
```

```
a = 7 && printf("Sharma");
```

```
a = 7 && 6
a = 1
```

```
!PankajSharma1
```

1:

```
int a;
a = printf("Pankaj") || printf("Sharma") && printf("Sir");
printf("%d",a);
```

a = printf("Pankaj") || (printf("Sharma") && printf("Sir"))

op1 op2

a = 6 || (x)

Pankaj 1

2:

```
int a;
a = printf("Pankaj") && printf("Sharma") || printf("Sir");
printf("%d", a);
```

a = (printf("Pankaj") && printf("Sharma")) || printf("Sir");

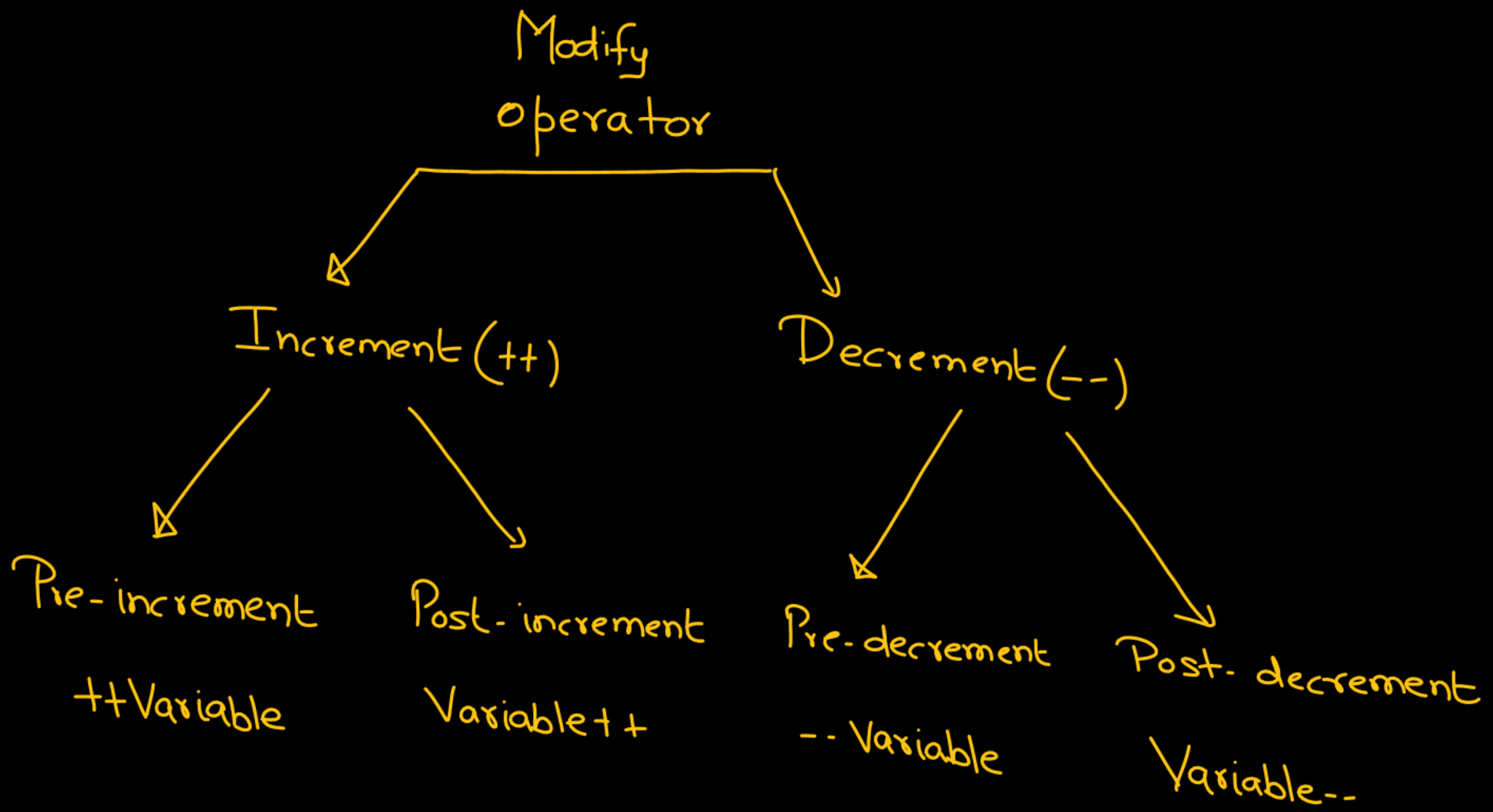
Op1 Op2

(6 && printf("Sharma")) || printf("Sir")

(6 && 6) || printf("Sir")

a = 1 || Pf("Sir")

PankajSharma1


Modify operator
Increment (++)
Decrement (--)
Pre-increment
++Variable
Post-increment
Variable++
Pre-decrement
--Variable
Post-decrement
Variable--

```
void main(){
    int a = 5;
    ++a;
    printf("%d",a);
}
```

a = a+1;

6;

a

~~7~~ 6

O/P : 6

a = a+1

```
Void main(){
    int a = 5;
    a++;
    printf("%d",a);
}
```

a

~~6~~

5;

a = a+1;

6

O/P : 6

```
void main(){
    int a=5,b;
    b=++a;
    printf("%d %d",a,b);
}
```

```
a=a+1;
b=a;
```

66

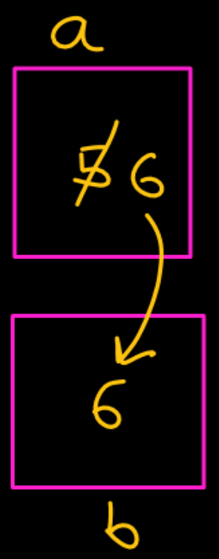
```
b=a;
a=a+1;
```

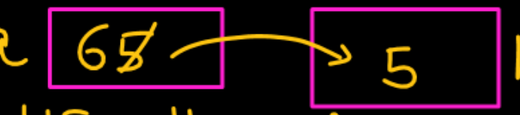
65

```
void main(){
    int a=5,b;
    b=a++;
    printf("%d %d",a,b);
}
```

(i) increase the value
a = a+1

(ii) Then use it
b=a





(i) Use the value of var.
b = a ;

(ii) Then increase
a = a+1 ;

```
int a=5,b;
b = a++ + ++a + ++a;
```

① ② ③

✓ result is not defined

Compiler dependent

Sequence point

{ You can modify the value of a variable atmost one time b/w 2 succ. seq. point. }

Bitwise Operators

(i) Bitwise AND (&)

(ii) Bitwise OR (|)

(iii) Bitwise XOR (^)

(iv) Bitwise left shift (<<)

(v) Bitwise right shift (>>)

(vi) Bitwise NOT (~)

binary

unary

Decimal Number System (10 Symbols)
0,1,2,3,4,5,6,7,8,9

Binary Number System (2 Symbols)
0/1

Octal Number System (8 Symbols)
0,1,2,3,4,5,6,7

Hexadecimal Number System (16 Symbols)
0-9, A, B, C, D, E, F

Decimal Number System

3

30

$3\ 4$
$10^1\ 10^0$ $\Rightarrow 3 \times 10^1 + 4 \times 10^0 = 30 + 4$

New Symbol

$3\ 4\ \boxed{5}$
$10^2\ 10^1\ 10^0$ $\Rightarrow 3 \times 10^2 + 4 \times 10^1 + 5$

$\underbrace{(3 \times 10^1 + 4 \times 10^0)} \times 10 + 5$

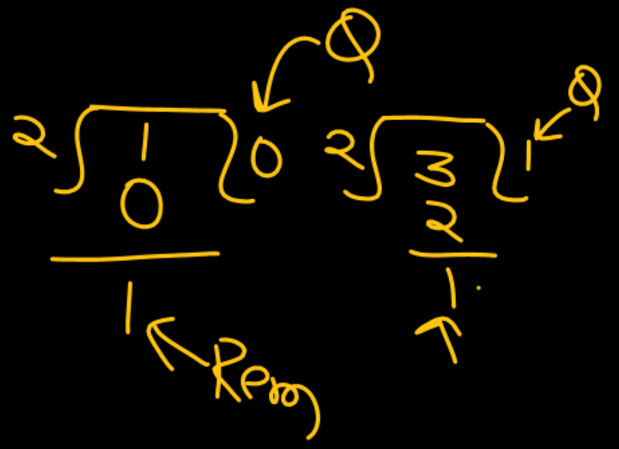
New value = old value $\times$ 10 + New Symbol

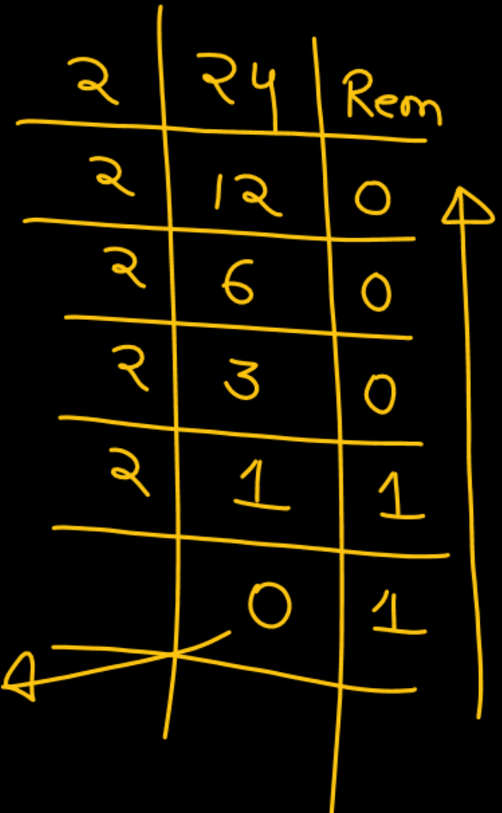
Binary Number System (0/1)

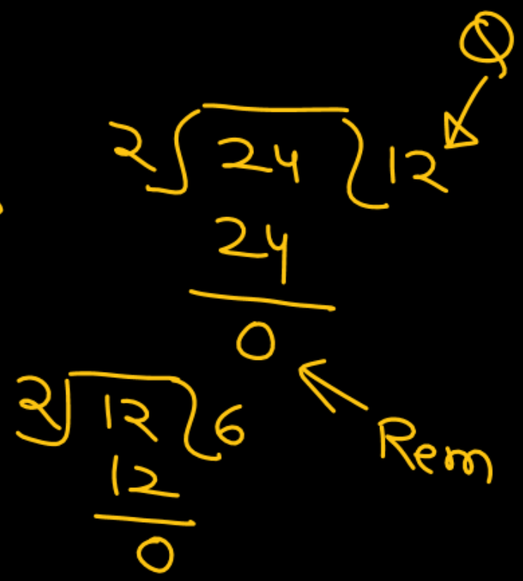
(i) Decimal ⟶ Binary

(ii) Binary ⟶ Decimal

$(24)_{10} = (11000)_2$







$(67)_{10} = (1000011)_2$

2	67	Rem
2	33	1
2	16	1
2	8	0
2	4	0
2	2	0
2	1	0
	0	1

Q

Binary → decimal

$$(11000)_2 = (\ ?\ )_{10}$$

$$\begin{matrix} 1 & 1 & 0 & 0 & 0 \\ 2^4 & 2^3 & 2^2 & 2^1 & 2^0 \end{matrix} \Rightarrow 1\times 2^4 + 1\times 2^3 + 0\times 2^2 + 0\times 2^1 + 0\times 2^0$$

$$= 16 + 8 + 0 + 0 + 0$$

$$= 24$$

$$\begin{array}{ccccccc} 1 & 0 & 0 & 0 & 0 & 1 & 1 \\ 2^6 & 2^5 & 2^4 & 2^3 & 2^2 & 2^1 & 2^0 \end{array}$$

$$\Rightarrow 1\times 2^6 + 0\times 2^5 + 0\times 2^4 + 0\times 2^3 + 0\times 2^2 + 1\times 2^1 + 1\times 2^0$$

$$\Rightarrow 64 + 0 + 0 + 0 + 0 + 2 + 1$$

$$= 67$$

value

10 → 2

10 0 ⇒

x

new value = $(x \times 2) + 0$

$= 2x$

10 0 → 4

1000 ⇒ 8 ✓

10 1 ⇒ $2 \times x + 1$

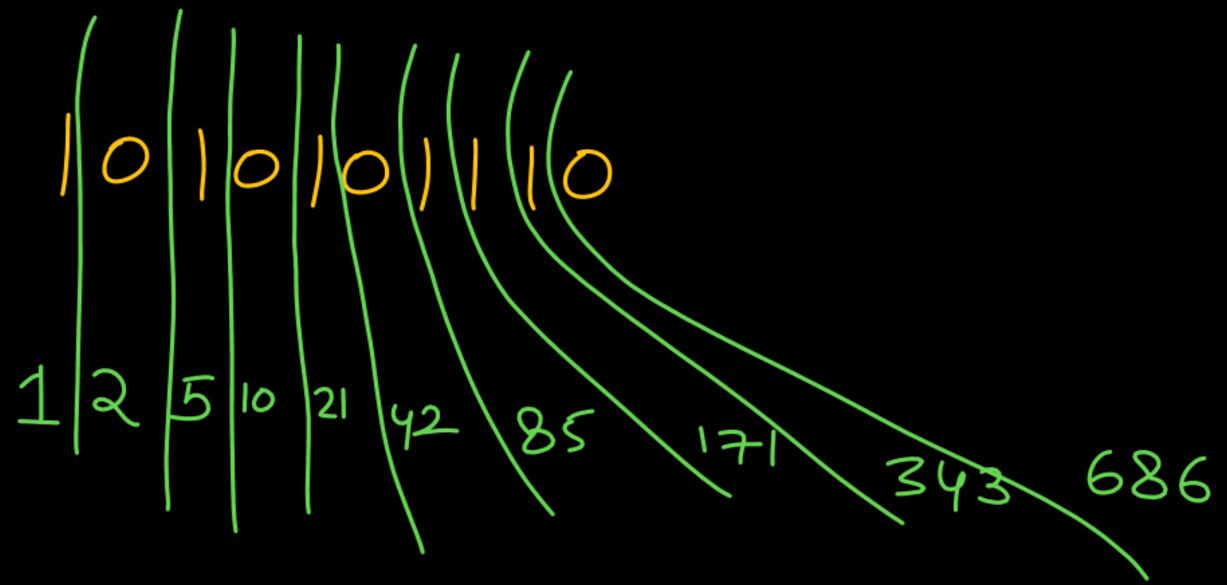
x

value

10 2

101 $2 \times 2 + 1 = 5$

1011 $2 \times 5 + 1 = 11$


1 0 1 0 1 0 1 1 1 0
1 2 5 10 21 42 85 171 343 686

① Bitwise AND(&)

```
int a=5,b=12,c
c=a&b;
printf("%d",c);
```

↳ (4)

a(5) : 0000 0000 0000 0101

b(12) : 0000 0000 0000 1100

c : 0000 0000 0000 0100

0 & 0 = 0

0 & 1 = 0

1 & 0 = 0

1 & 1 = 1

⇒ c ⇒ 4

② Bitwise OR (|):

```
int a = 5, b = 12, c;
c = a | b ;
```

```
a : 0000 0000 0000 0101
b : 0000 0000 0000 1100
c : 0000 0000 0000 1101
```

0 | 0 = 0

1 | 0 = 1

0 | 1 = 1

1 | 1 = 1

1101

~~1361~~

1101

$2^3 2^2 2^1 2^0$

$\Rightarrow 1\times2^3 + 1\times2^2 + 0\times2^1 + 1\times2^0$

$= 8 + 4 + 0 + 1$

$= 13$

③ Bitwise XOR(^) :

0 ^ 0 = 0 } both bits are same
1 ^ 1 = 0 }

0 ^ 1 = 1 } both bits are different
1 ^ 0 = 1 }

```
int a = 5, b = 12, c;
c = a ^ b;
printf("%d", c);
```

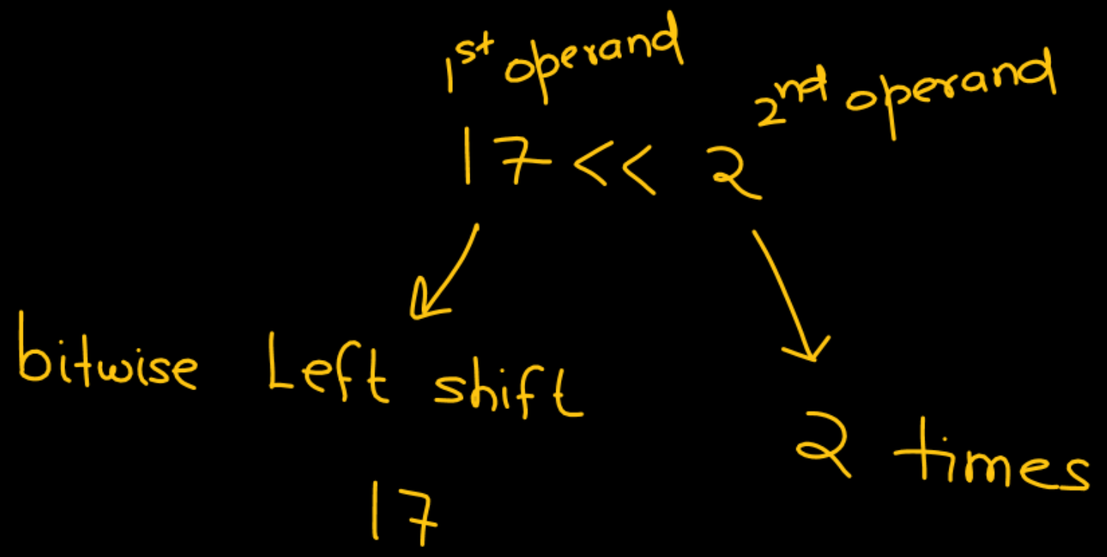
↳ 9

a : 0000 0000 0000 0101
b : 0000 0000 0000 1100
---
c : 0000 0000 0000 1001

↳ 9

Bitwise left-shift (<<)

Binary operator



19 << 3

left shift 19, 3 times

```
int a;
a = 10;
```

```
printf("%d", a<<1);
```

⇓

20

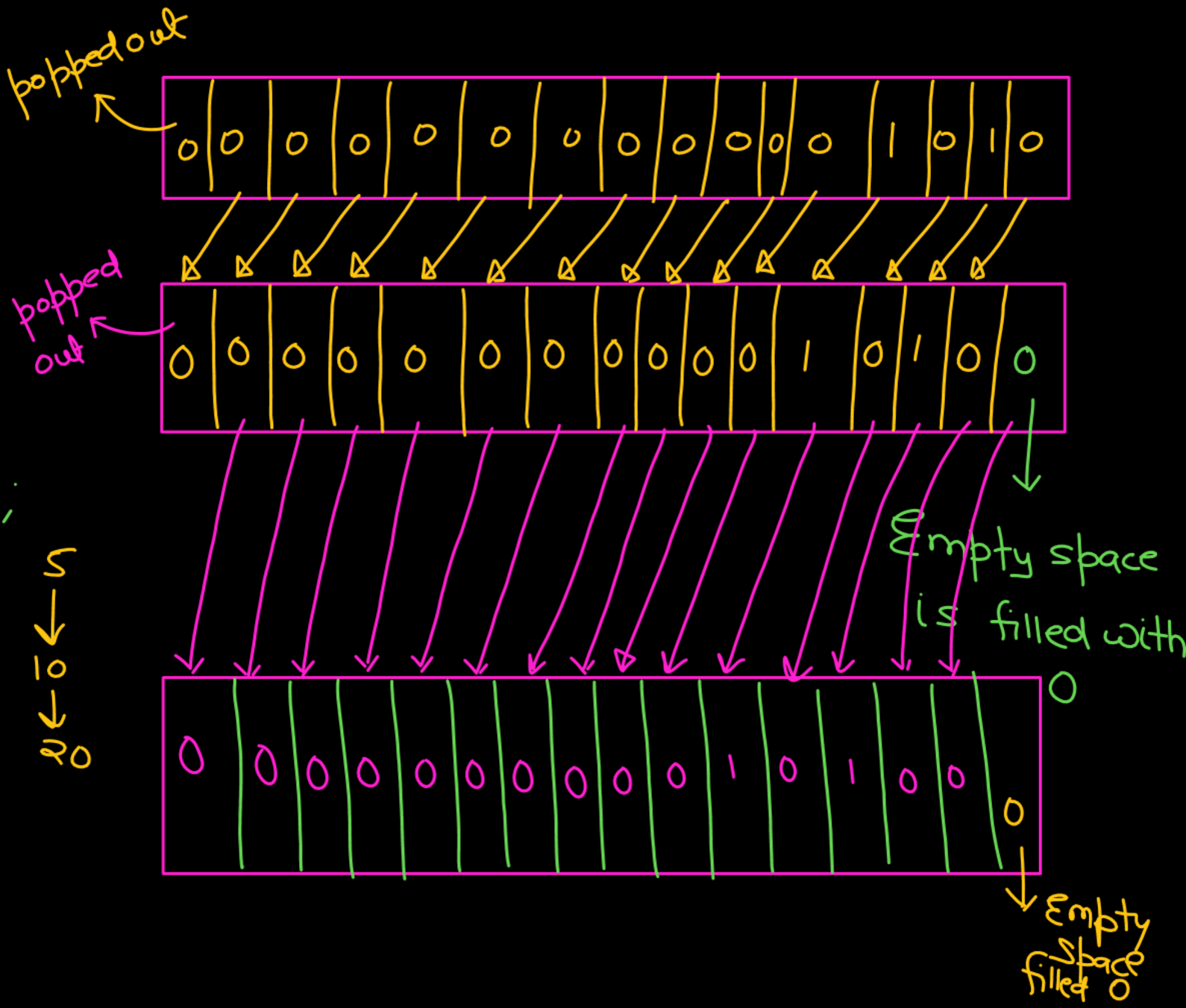
```
printf("%d", a<<2);
```

16 << 2

⇒ 16 × 2 × 2

⇒ 16 × 2^2

= (64)



Bitwise Right shift (>>)

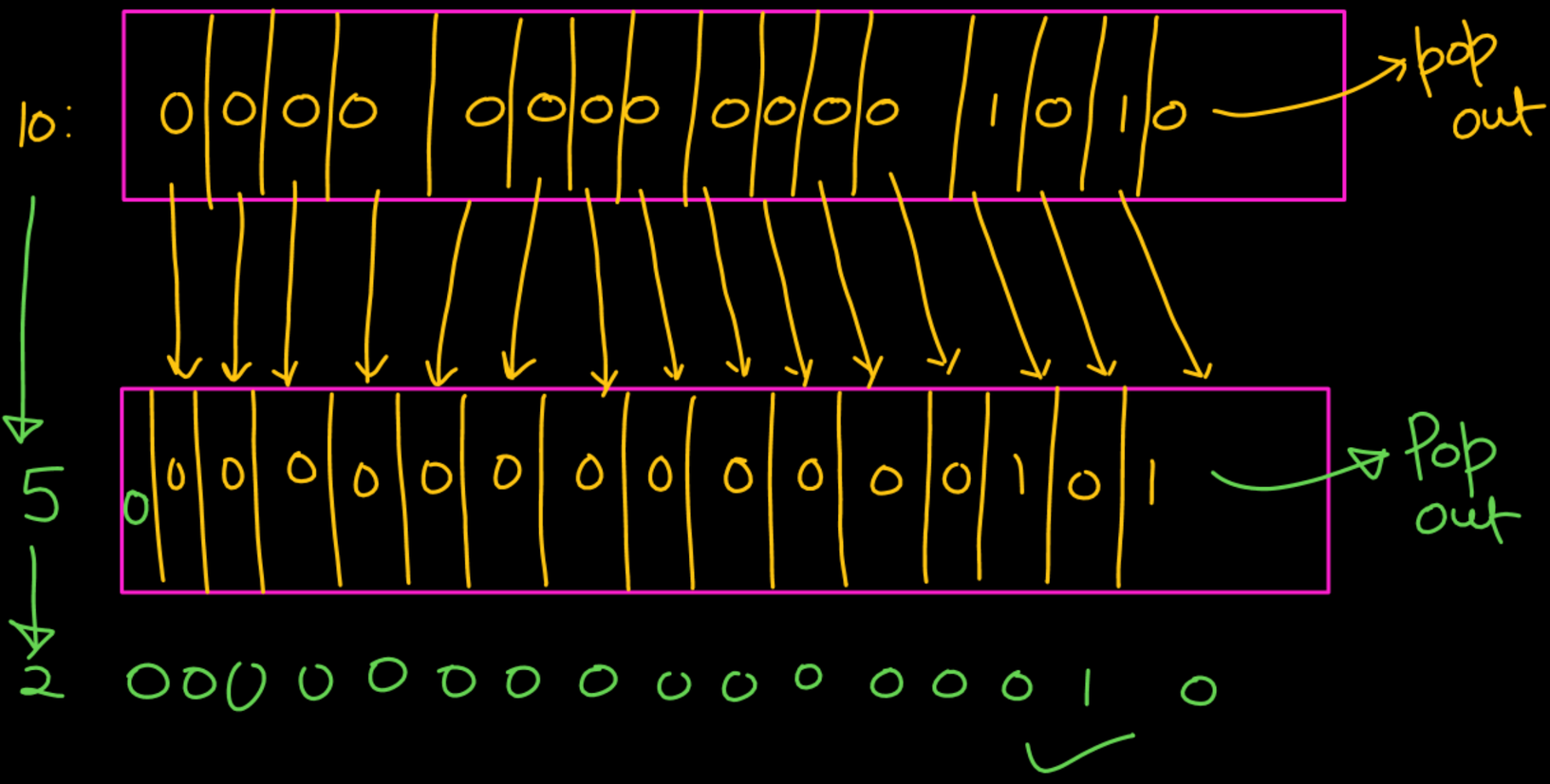
17 >>1

⇒ Right shift 17 by 1 time

19 >>2

⇒ Right shift 19 by 2 times

```
int a = 10;
printf("%d", a>>1);
printf("%d", a>>2);
```



$10 / 2 = 5$ r 0

$5 / 2 = 2$ r 1

$10 >> 2$

$\Rightarrow \frac{10}{2^2} = \frac{10}{4} = 2$


THANK YOU GW
SOLDIERS !